Высшая ИТ-школа

КГУ Костромской государственный университет

2

Arduino. Модуль 2: виртуальная лаборатория

Симулятор · Экран LCD · Датчики · Сервопривод · Дальномер · millis() · Хакатон

Занятия 9–16 · Cirkit Designer

Что мы уже умеем

Высшая ИТ-школа

1

Оживить плату

`setup`, `loop`, `pinMode`, `digitalWrite`, `delay` — мигалка Blink

2

Собрать цепь

Макетка, светодиод, резистор, переменные `int` — сигнал SOS

3

Слушать мир

Кнопка, баззер, `if / else`, `tone()` — телеграф

4

Укрощать хаос

Массивы и цикл `for` — бегущий огонь

5

Аналог и ШИМ

Потенциометр, `analogRead`, `analogWrite`, `map()`

6

Умный дом

Датчик звука и переменная состояния — хлопковый ночник

7

Безопасность

RFID, SPI, библиотеки, RGB — «свой-чужой»

8

Финал

Проекты из нескольких модулей: сейф, диско-лампа, замок

План модуля 2

Высшая ИТ-школа

День	Тема	Миссия
9	Виртуальная лаборатория: переезд в Cirkit Designer	**Светофор с кнопкой**
10	Свои команды: функции	**Электронный кубик**
11	Робот заговорил: экран LCD 16×2	**Табло-счётчик**
12	Метеостанция: температура и влажность	**Метеостанция с RGB**
13	Движение: сервопривод	**Шлагбаум**
14	Летучая мышь: ультразвуковой дальномер	**Парктроник**
15	Многозадачность: millis() вместо delay()	**Парктроник 2.0**
16	Финальный хакатон	**Теплица · Гараж · Сейф 2.0**

Финальная цель: три проекта «умного города»

Высшая ИТ-школа

1

Умная теплица

Датчик температуры, форточка на сервоприводе, экран и RGB. Жарко — форточка сама открывается.

2

Умный гараж

Дальномер видит машину и поднимает шлагбаум. Парктроник пищит, если подъехали слишком близко.

3

Сейф 2.0

Клавиатура 4×4, код на экране звёздочками, серво-замок и сирена после трёх ошибок.

Высшая ИТ-школа

КГУ Костромской государственный университет

09

Виртуальная лаборатория: переезд в Cirkit Designer

Cirkit Designer

INPUT_PULLUP

== != > <

Монитор порта

Светофор

Занятие 9

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Чем симулятор отличается от **настоящей платы**
- Как устроен **Cirkit Designer**: Design, Code, Simulate
- Как читать кнопку без резистора: `INPUT_PULLUP`
- Как сравнивать числа: `==`, `!=`, `>`, `<`
- Как смотреть, что думает плата: **монитор порта**

✓ В результате урока

- Соберёте первую схему **в браузере**
- Перенесёте туда Blink из занятия 1
- Запустите **светофор с кнопкой** для пешехода и звуковым сигналом

Настоящая плата VS симулятор

Высшая ИТ-школа

Настоящая плата

- Чувствуешь детали руками
- Можно взять домой и показать
- Деталь можно **сжечь**
- Набор деталей ограничен
- Ошибку в проводах искать долго

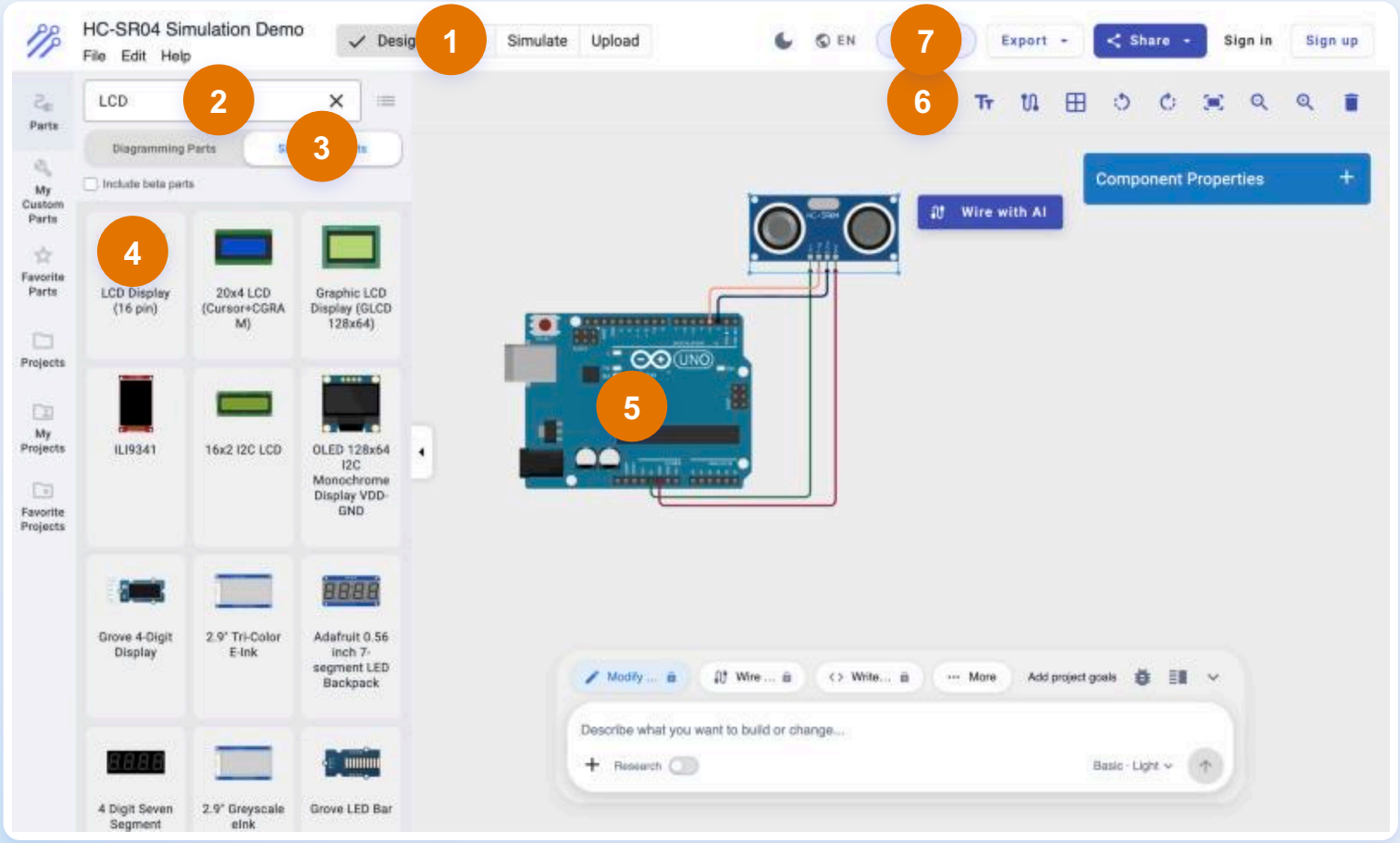
VS

Симулятор Cirkit Designer

- Сотни деталей: экраны, датчики, моторы
- Ничего не сгорит — можно экспериментировать
- Работает в браузере, дома тоже
- Код — тот же язык Arduino C++
- Правила инженера — **те же самые**

Интерфейс: вкладка Design (схема)

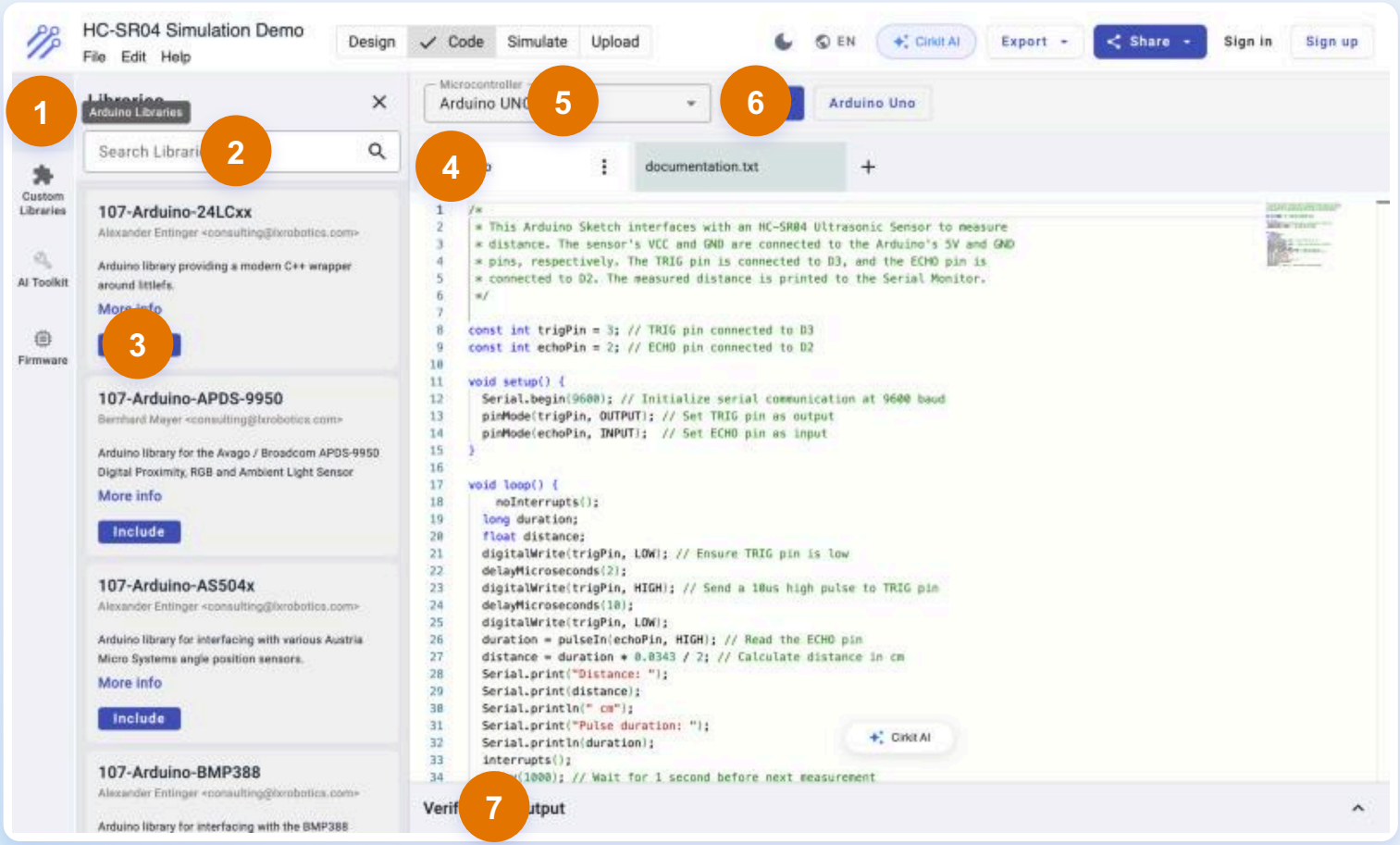
Высшая ИТ-школа



1. Вкладки **Design** (схема) · **Code** (код) · **Simulate** (запуск)
2. **Поиск деталей**: LED, Pushbutton, Resistor…
3. **Simulation Parts** — только детали, которые оживают в симуляции
4. Карточки деталей: **перетащите** нужную на поле
5. Рабочее поле: здесь собираем схему
6. **Play** — запуск симуляции
7. **Cirkit AI** — помощник (только с разрешения!)

Интерфейс: вкладка Code (код)

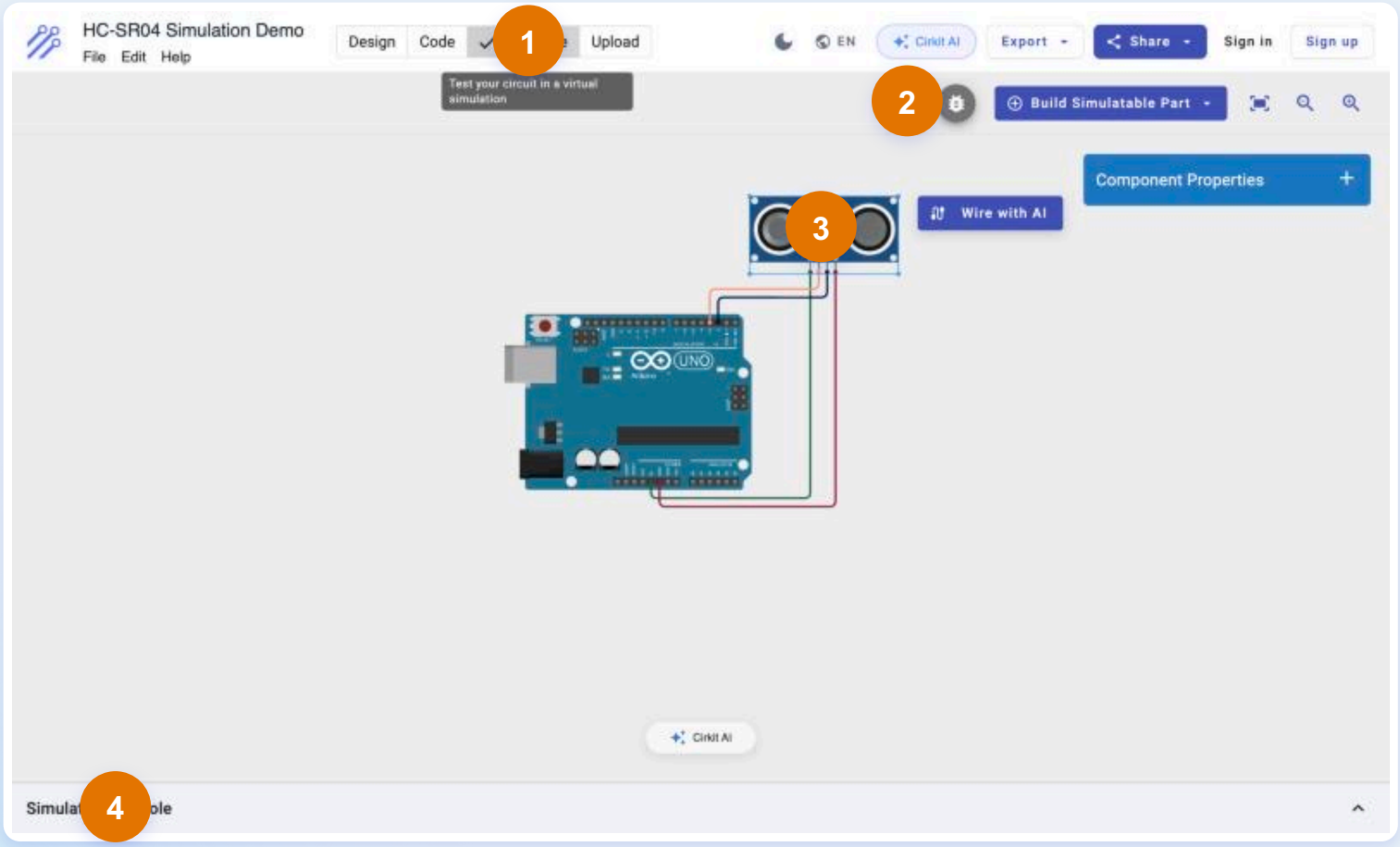
Высшая ИТ-школа



1. **Libraries** — библиотеки (понадобятся с занятия 11)
2. Поиск библиотеки по названию
3. **Include** — подключить библиотеку к проекту
4. **sketch.ino** — наш скетч, как в Arduino IDE
5. Выбор платы: **Arduino UNO**
6. **Verify** — проверить код (галочка из Arduino IDE)
7. **Verification Output** — здесь будут ошибки

Интерфейс: вкладка Simulate (запуск)

Высшая ИТ-школа



1. Вкладка **Simulate**
2. **Play / Stop** — запустить или остановить
3. Во время симуляции кнопки **нажимаются мышкой**, у датчиков меняются значения
4. **Simulation Console** — наш монитор порта

Как собрать схему в Cirkit: 5 шагов

Высшая ИТ-школа

1

Плата

Найдите **Arduino UNO** и перетащите на поле

2

Детали

Добавьте детали из **Simulation Parts**: LED, Resistor, Pushbutton

3

Провода

Кликните по выводу детали и протяните провод к пину платы

4

Код

Вкладка **Code** → пишем скетч → **Verify**

5

Запуск

Вкладка **Simulate** → **Play**. Не работает? Stop → ищем ошибку

Договоримся о цветах проводов: красный — 5V, чёрный — GND, остальные — сигналы. Так любую схему можно проверить за 10 секунд.

Правила виртуальной лаборатории

Высшая ИТ-школа

1

Сохраняйся

Имя проекта: **Фамилия_Занятие9**. Сохраняем после каждого успешного шага.

2

Только Simulation Parts

Деталь из Diagramming Parts не оживёт — схема молча не заработает.

3

Сначала сам

Cirkit AI — только после **двух своих попыток** и с разрешения преподавателя.

4

Цвета проводов

Красный — 5V, чёрный — GND. Схема должна читаться с одного взгляда.

5

Схема → код → Play

Не жмём Play, пока не проверили провода. Привычка инженера!

6

Помогаем друг другу

Глупых вопросов не бывает. Нашёл ошибку у соседа — подскажи, но не делай за него.

Разминка: Blink в симуляторе

Высшая ИТ-школа

sketch.ino

```
int ledPin = 10;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  delay(500);
  digitalWrite(ledPin, LOW);
  delay(500);
}
```

1 Схема

Светодиод: длинная ножка (+) → **D10**, короткая (−) → резистор 220 Ом → **GND**

2 Проверка

Verify → вкладка **Simulate** → **Play**. Светодиод мигает?

3 Эксперимент

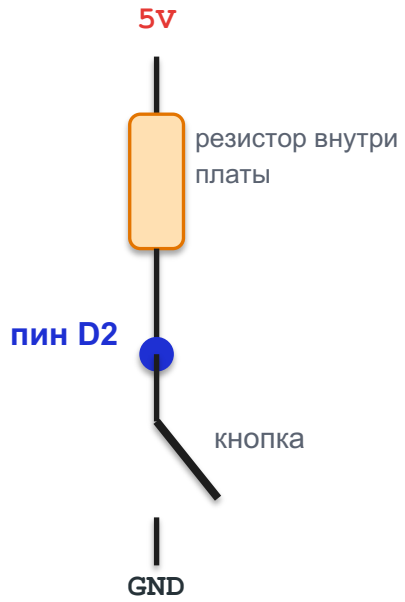
Поменяйте `500` на `100`. А на `2000`?

Кнопка без резистора: INPUT_PULLUP

Высшая ИТ-школа

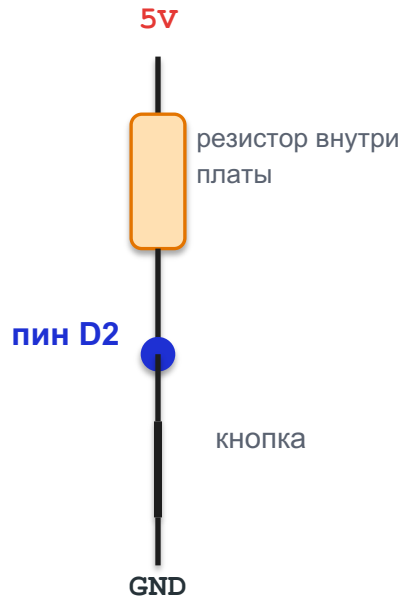
- Внутри платы есть **свой резистор**, который тянет пин к 5V
- Включается так: `pinMode(2, INPUT_PULLUP);`
- Кнопку подключаем между **пином** и **GND** — и всё!
- Логика **перевёрнута**: не нажата → `HIGH`, нажата → `LOW`
- Помните ошибку из занятия 3 «пищит, а при нажатии молчит»? Вот её причина

Кнопка НЕ нажата



digitalRead = HIGH

Кнопка нажата



digitalRead = LOW

Сравнения: как плата задаёт вопросы

Высшая ИТ-школа

`==`	**равно**	`button == LOW`
`!=`	**не равно**	`count != 0`
`>`	**больше**	`temp > 28`
`<`	**меньше**	`cm < 10`
`>=`	**больше или равно**	`count >= 3`
`<=`	**меньше или равно**	`t <= 26`

Ловушка! `=` и `==` — разные вещи.

`count = 5` — **положить** 5 в коробку. `count == 5` — **спросить**: в коробке 5?

Монитор порта: рентген для кода

Высшая ИТ-школа

test_button.ino

```
void setup() {
  pinMode(2, INPUT_PULLUP);
  Serial.begin(9600);   // открываем связь
}

void loop() {
  int button = digitalRead(2);
  Serial.println(button);  // печатаем 1 или 0
  delay(200);
}
```

1. **Serial.begin(9600)**

   Открывает канал связи. Пишем один раз в `setup`.

2. **Serial.println(…)**

   Печатает число или текст и переходит на новую строку.

3. **Где смотреть?**

   В Cirkit — внизу, в **Simulation Console**.

4. **Зачем?**

   Видно, что «думает» плата. Это главный инструмент поиска ошибок.

Миссия: светофор с кнопкой для пешехода

Высшая ИТ-школа

МИССИЯ

1. Машинам всё время горит **зелёный**
2. Пешеход нажимает кнопку → **жёлтый** на 2 секунды
3. Затем **красный** на 5 секунд, баззер пищит каждые полсекунды — сигнал для незрячих пешеходов
4. Снова зелёный. В мониторе порта — сообщение «Пешеход нажал кнопку»

Готово, если:

- Зелёный горит без нажатия
- Кнопка запускает цикл жёлтый → красный
- Слышен писк во время красного
- Сообщение в Simulation Console

★ Усложнение

Добавьте пешеходный светофор: 2 светодиода (красный и зелёный для пешехода), которые горят «наоборот» машинному.

Карта подключения: светофор

Высшая ИТ-школа

Как собрать

1. Arduino UNO + **Half Breadboard**
2. Синюю линию макетки соедините с **GND** — это общая шина
3. Три светодиода: короткая ножка → резистор → шина GND
4. Длинные ножки → D10, D11, D12
5. Кнопка: один вывод в D2, другой в шину GND
6. Баззер (Piezo): «+» в D8, «−» в шину GND

Код светофора: настройка

Высшая ИТ-школа

svetofor.ino

```
int redPin = 10;
int yellowPin = 11;
int greenPin = 12;
int buttonPin = 2;
int buzzerPin = 8;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  Serial.begin(9600);
  Serial.println("Светофор запущен");
}
```

1. **Переменные-коробочки**

   Номера пинов храним в переменных — как на занятии 2.

2. **INPUT_PULLUP**

   Кнопка без внешнего резистора.

3. **Serial**

   Открываем монитор порта и здороваемся.

Код светофора: главный цикл

Высшая ИТ-школа

svetofor.ino

```
void loop() {
  digitalWrite(greenPin, HIGH);

  if (digitalRead(buttonPin) == LOW) {
    Serial.println("Пешеход нажал кнопку");
    digitalWrite(greenPin, LOW);
    digitalWrite(yellowPin, HIGH);
    delay(2000);
    digitalWrite(yellowPin, LOW);
    digitalWrite(redPin, HIGH);
    for (int i = 0; i < 10; i++) {
      tone(buzzerPin, 1000, 100);
      delay(500);
    }
    digitalWrite(redPin, LOW);
  }
}
```

1 **== LOW**

Кнопку нажали — пин соединился с GND.

2 **for 10 раз**

10 писков по 0,5 с = 5 секунд красного.

3 **tone(пин, 1000, 100)**

Третье число — длительность писка в мс.

Дебаггинг: ищем ошибки

Высшая ИТ-школа

Кнопка ничего не делает	Проверьте: второй вывод кнопки идёт в **GND**, а в коде `INPUT_PULLUP` и `== LOW`.
Светофор переключается сам, без кнопки	Забыли `INPUT_PULLUP` — пин «висит в воздухе» и ловит помехи.
Светодиод не горит	Перевёрнут (длинная ножка — к пину) или деталь не из **Simulation Parts**.
Красная ошибка после Verify	Ищем строку из Verification Output: пропущенная `;` или `}`, `=` вместо `==`.
Play не запускается	Сначала **Verify** без ошибок. Проверьте, что выбрана плата **Arduino UNO**.

Итоги и рефлексия

Высшая ИТ-школа

Сегодня мы:

- Переехали в **Cirkit Designer**: Design → Code → Simulate
- Подключили кнопку без резистора через `INPUT_PULLUP`
- Разобрались: `=` кладёт, `==` спрашивает
- Научились смотреть в **монитор порта**
- Запустили **светофор** с кнопкой и звуком

1. **Что вам понравилось в работе?**

2. **Что нового было самым важным?**

3. **Что вызвало наибольшие сложности?**

Высшая ИТ-школа

КГУ Костромской государственный университет

10

Свои команды: функции и электронный кубик

void | Параметры | return | random() | 7 сегментов | Кубик

Занятие 10

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Что такое **функция** и зачем она нужна
- Как создать свою команду: `void имя() { … }`
- Как передать функции **параметр**
- Как функция **возвращает** ответ: `return`
- Как устроен **7-сегментный индикатор**

✓ В результате урока

- Перепишете SOS в 4 строчки
- Соберёте **электронный игральный кубик**: кнопка → анимация → случайное число от 1 до 6

Проблема: код повторяется

Высшая ИТ-школа

sos_old.ino

```
// буква S — три точки
digitalWrite(ledPin, HIGH); delay(200);
digitalWrite(ledPin, LOW);  delay(200);
digitalWrite(ledPin, HIGH); delay(200);
digitalWrite(ledPin, LOW);  delay(200);
digitalWrite(ledPin, HIGH); delay(200);
digitalWrite(ledPin, LOW);  delay(200);
// буква O — три тире
digitalWrite(ledPin, HIGH); delay(600);
digitalWrite(ledPin, LOW);  delay(200);
// ... и ещё 12 строк ...
```

1. **Много одинакового**

   SOS из занятия 2 занимал 30+ строк.

2. **Легко ошибиться**

   Пропустил одну строку — и сигнал сломан.

3. **Трудно читать**

   Где тут буква S, а где O?

4. **Решение**

   Дать куску кода **имя** и вызывать его по имени.

Функция — это рецепт с названием

Высшая ИТ-школа

- Функция — кусок кода, у которого есть **имя**
- Написали **один раз** — вызываем сколько угодно
- `void` — функция просто делает дело и ничего не отвечает
- Функции пишем **вне** `setup` и `loop` — обычно над ними
- Вызов: имя и скобки — `dot();`

```
void dot() {
  digitalWrite(ledPin, HIGH);
  delay(200);
  digitalWrite(ledPin, LOW);
  delay(200);
}
```

void — ничего не возвращает

dot — имя — придумываем сами, латиницей

{ ... } — тело — что делать

SOS через функции

Высшая ИТ-школа

sos_functions.ino

```
void dot() {
  digitalWrite(ledPin, HIGH); delay(200);
  digitalWrite(ledPin, LOW);  delay(200);
}
void dash() {
  digitalWrite(ledPin, HIGH); delay(600);
  digitalWrite(ledPin, LOW);  delay(200);
}
void letterS() { dot();  dot();  dot();  }
void letterO() { dash(); dash(); dash(); }

void loop() {
  letterS();
  letterO();
  letterS();
  delay(2000);
}
```

1. **dot и dash**

   Точка и тире — наши «кирпичики».

2. **letterS и letterO**

   Функции могут вызывать **другие функции**.

3. **loop**

   Читается как текст: S, O, S, пауза.

Функция с параметром

Высшая IT-школа

params.ino

```
void blinkTimes(int n) {
  for (int i = 0; i < n; i++) {
    digitalWrite(ledPin, HIGH);
    delay(300);
    digitalWrite(ledPin, LOW);
    delay(300);
  }
}

void loop() {
  blinkTimes(3);    // мигнуть 3 раза
  delay(1000);
  blinkTimes(5);    // мигнуть 5 раз
  delay(1000);
}
```

1 **int n**

Параметр — коробочка, которую функция получает при вызове.

2 **blinkTimes(3)**

Внутри функции `n` станет равным 3.

3 **Как кофемашина**

Кнопка одна, а сколько сахара — решаешь при нажатии.

Функция, которая отвечает: return

Высшая ИТ-школа

dice_serial.ino

```
int rollDice() {
  return random(1, 7);  // от 1 до 6
}

void loop() {
  int result = rollDice();
  Serial.println(result);
  delay(1000);
}
```

1 **int вместо void**

Функция **вернёт** целое число.

2 **return**

«Вот ответ!» — функция отдаёт число и заканчивается.

3 **random(1, 7)**

Случайное число от 1 до **6**: верхняя граница не входит!

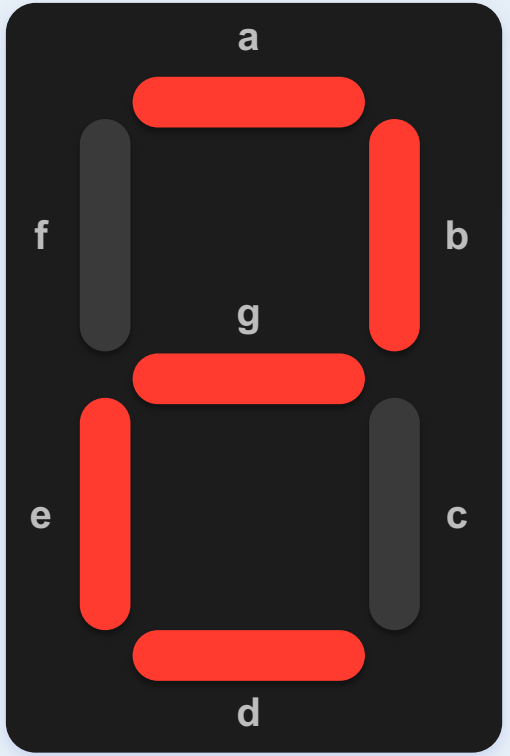
4 **Где уже видели?**

`analogRead(A0)` и `digitalRead(2)` — тоже возвращают ответ.

Знакомьтесь: 7-сегментный индикатор

Высшая ИТ-школа

- Это **7 светодиодов-палочек** в одном корпусе
- Каждый сегмент назван буквой: **a, b, c, d, e, f, g**
- Зажигая разные сегменты, рисуем любую цифру
- Общий вывод **COM**: общий катод → GND, общий анод → 5V
- Такие индикаторы — в часах, микроволновках, лифтах



Цифра **2**:

горят **a, b, g, e, d**

не горят: c, f

Таблица цифр: массив массивов

Высшая ИТ-школа

- Для каждой цифры храним **7 нулей и единиц**
- 1 — сегмент горит, 0 — не горит
- Это **двумерный массив**: строки — цифры, столбцы — сегменты
- `digits[2][0]` — цифра 2, сегмент a → **1**
- Как в «Морском бое»: строка + столбец = клетка

	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1

…и так до 9

Карта подключения: кубик

Высшая ИТ-школа

Как собрать

1. Найдите в Simulation Parts **Seven Segment Display**
2. Наведите на выводы — Cirkit подскажет их имена (a…g, COM)
3. Сегменты a…g → пины D4…D10 **по порядку**
4. COM → GND (если индикатор с общим анодом → 5V)
5. Кнопка: D2 и GND
6. Точку (DP) не подключаем

Код кубика: функции

Высшая ИТ-школа

kubik.ino

```
int segPins[7] = {4, 5, 6, 7, 8, 9, 10};
const int ON = HIGH;   // общий анод? -> LOW
const int OFF = LOW;   // общий анод? -> HIGH

int digits[10][7] = {
  {1,1,1,1,1,1,0}, {0,1,1,0,0,0,0},   // 0, 1
  {1,1,0,1,1,0,1}, {1,1,1,1,0,0,1},   // 2, 3
  {0,1,1,0,0,1,1}, {1,0,1,1,0,1,1},   // 4, 5
  {1,0,1,1,1,1,1}, {1,1,1,0,0,0,0},   // 6, 7
  {1,1,1,1,1,1,1}, {1,1,1,1,0,1,1}    // 8, 9
};

void showDigit(int n) {
  for (int s = 0; s < 7; s++) {
    if (digits[n][s] == 1) digitalWrite(segPins[s], ON);
    else digitalWrite(segPins[s], OFF);
  }
}
int rollDice() { return random(1, 7); }
```

1. **const**

   Константа — коробочка, которую **нельзя менять**.

2. **showDigit(n)**

   Функция с параметром: рисует цифру n.

3. **rollDice()**

   Функция-ответчик: возвращает 1…6.

Код кубика: setup и loop

Высшая ИТ-школа

kubik.ino

```ino
void setup() {
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
  pinMode(2, INPUT_PULLUP);
  randomSeed(analogRead(A0));
  showDigit(0);
}

void loop() {
  if (digitalRead(2) == LOW) {
    for (int i = 0; i < 10; i++) {  // кубик катится
      showDigit(rollDice());
      delay(80);
    }
    showDigit(rollDice());           // итог
    delay(300);
  }
}
```

1. **for в setup**

   Настраиваем 7 пинов одним циклом.

2. **randomSeed**

   «Перемешивает колоду», чтобы числа не повторялись при каждом запуске.

3. **Анимация**

   10 быстрых случайных цифр — эффект катящегося кубика.

Миссия: электронный кубик

Высшая ИТ-школа

МИССИЯ

1. Соберите индикатор и кнопку по карте
2. Проверьте `showDigit`: в `setup` покажите по очереди 0…9
3. Напишите `rollDice()` и анимацию в `loop`
4. Бросьте кубик 10 раз — проверьте, что выпадают **все числа 1–6**

Готово, если:

- Цифры рисуются правильно
- Кнопка запускает анимацию
- Итог — от 1 до 6
- В коде есть свои функции

★ Усложнение

Кубик для «Монополии»: две кнопки — одна бросает кубик 1–6, другая — «счастливая» 1–9. Или: после броска баззер пищит столько раз, сколько выпало.

Дебаггинг: ищем ошибки

Высшая ИТ-школа

Цифры «наоборот»: горит всё, кроме нужного	Индикатор с **общим анодом**: COM → 5V, а в коде поменяйте `ON = LOW`, `OFF = HIGH`.
Цифры похожи на иероглифы	Перепутан порядок пинов: a → D4, b → D5 … g → D10. Проверьте по карте.
Всегда выпадает одно и то же	Нет `randomSeed` или анимация не вызывается — проверьте `loop`.
Ошибка: 'showDigit' was not declared	Опечатка в имени функции: большие и маленькие буквы различаются!

Итоги и рефлексия

Высшая ИТ-школа

Сегодня мы:

- Узнали, что такое **функция** и зачем она нужна
- Научились передавать **параметры** и возвращать ответ через `return`
- Разобрали **двумерный массив** — таблицу цифр
- Собрали **электронный кубик** на 7-сегментном индикаторе

1. **Что вам понравилось в работе?**
2. **Что нового было самым важным?**
3. **Что вызвало наибольшие сложности?**

Высшая ИТ-школа

КГУ Костромской государственный университет

11

Робот заговорил: экран LCD 16×2

LCD 16×2 | I2C | Библиотеки | setCursor | Счётчик нажатий

Занятие 11

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Как устроен **экран LCD 16×2**
- Что такое шина **I2C** и чем она удобнее
- Как подключить **библиотеку** в Cirkit
- Как поставить курсор и напечатать текст
- Как поймать **момент** нажатия кнопки

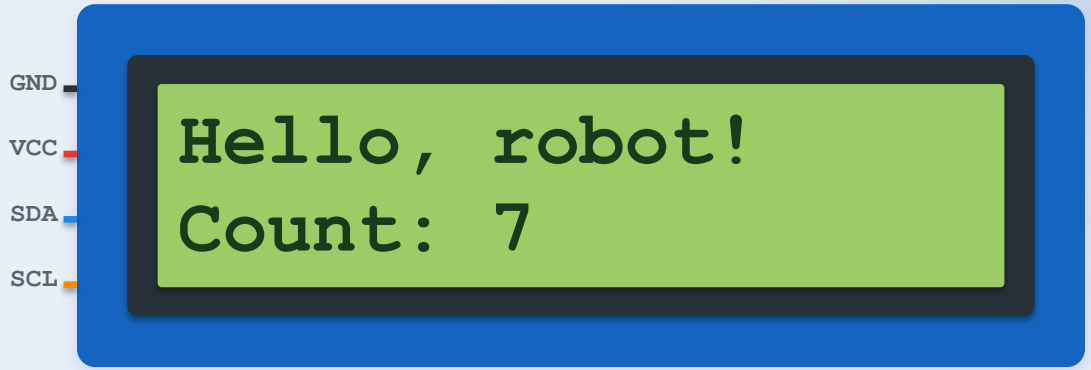
✓ В результате урока

- Соберёте **табло**: счётчик нажатий кнопки и шкалу-индикатор от потенциометра

Знакомьтесь: экран LCD 16×2

Высшая ИТ-школа

- **16** символов в строке × **2** строки = 32 клетки
- Каждая клетка — маленькая сетка **5×8 точек**
- Модуль **I2C** на обратной стороне: всего **4 провода**
- Подсветку можно включать и выключать из кода
- Такие экраны — в кассах, принтерах, 3D-принтерах



Шина I2C: 4 провода вместо 16

Высшая ИТ-школа

SPI (занятие 7, RFID)

- Отдельные провода: MOSI, MISO, SCK, SS
- Очень быстрая
- Каждому устройству — свой провод SS
- 7 проводов для сканера

VS

I2C (сегодня, экран)

- Всего 2 сигнальных провода:
- **SDA** — данные (на Uno это **A4**)
- **SCL** — метроном (на Uno это **A5**)
- У каждого устройства свой **адрес**, например `0x27`
- Можно повесить много устройств на одни провода

Подключаем библиотеку в Cirkit

Высшая ИТ-школа

1

Code

Откройте вкладку **Code**

2

Libraries

Слева нажмите **Libraries**

3

Поиск

Введите **LiquidCrystal I2C**

4

Include

У библиотеки **LiquidCrystal I2C** (Marco Schwartz) нажмите **Include**

5

Проверка

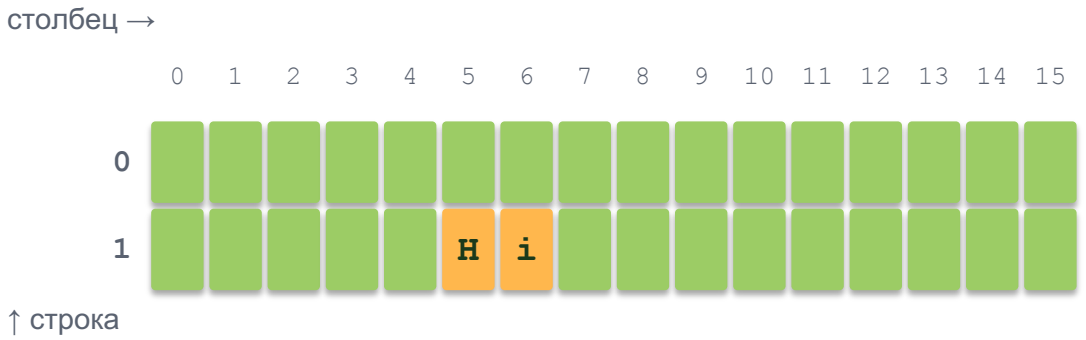
В начале скетча появится `#include` — жмём **Verify**

Библиотека = готовый словарь команд. Подключили один раз — и плата понимает `lcd.print()`, `lcd.setCursor()` и другие слова.

Главные команды экрана

Высшая ИТ-школа

- `lcd.init();` — разбудить экран (в `setup`)
- `lcd.backlight();` — включить подсветку
- `lcd.setCursor(5, 1);` — **столбец 5, строка 1**
- `lcd.print("Hi");` — напечатать текст или число
- `lcd.clear();` — стереть всё
- Счёт снова с **нуля**: столбцы 0…15, строки 0…1



`setCursor(5, 1)` + `print("Hi")`

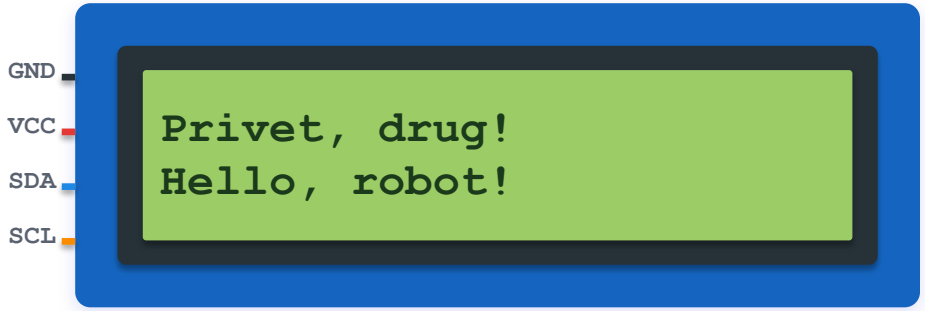
→ текст с 5-й клетки второй строки

Две ловушки экрана

Высшая ИТ-школа

1 Экран не знает русских букв

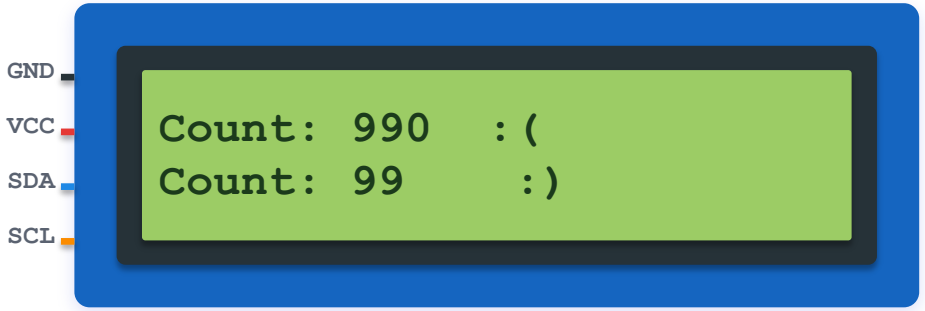
Встроенный шрифт — только латиница и цифры. Пишем `"Privet"`, а не «Привет».



2 Старые символы не стираются

Было `100`, стало `99` → на экране `990`. Лечится пробелами после числа:

```
lcd.print(count); lcd.print("  ");
```



Ловим момент нажатия

Высшая ИТ-школа

- `loop` крутится тысячи раз в секунду
- Пока палец на кнопке, `digitalRead` много раз подряд даёт `LOW`
- Если считать каждый `LOW` — одно нажатие = **сотни** очков
- Решение: помнить, какой кнопка была **в прошлый раз** — `lastButton`
- Считаем, только когда было `HIGH`, а стало `LOW`

```
if (button == LOW && lastButton == HIGH) {
  count = count + 1;
}
lastButton = button;
```

Карта подключения: табло

Высшая ИТ-школа

Как собрать

1. Найдите **16x2 I2C LCD** в Simulation Parts
2. SDA → **A4**, SCL → **A5** (только эти пины!)
3. Потенциометр (Trimmer Potentiometer): края — 5V и GND, середина — A0
4. Кнопка — D2 и GND (`INPUT_PULLUP`)
5. 5V и GND удобно вывести на шину макетки

Код табло: подготовка

Высшая IT-школа

tablo.ino

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

int buttonPin = 2;
int count = 0;
int lastButton = HIGH;

void setup() {
  pinMode(buttonPin, INPUT_PULLUP);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Hello, robot!");
  delay(1000);
  lcd.clear();
}
```

1. **#include**

   Подключаем словари: Wire (I2C) и экран.

2. **lcd(0x27, 16, 2)**

   Создаём экран: адрес 0x27, 16 символов, 2 строки.

3. **lastButton**

   Память кнопки: в начале она не нажата (HIGH).

4. **Приветствие**

   Через секунду экран очищается — начинаем работу.

Код табло: главный цикл

Высшая IT-школа

tablo.ino

```
void loop() {
  int button = digitalRead(buttonPin);
  if (button == LOW && lastButton == HIGH) {
    count = count + 1;
  }
  lastButton = button;

  int bar = map(analogRead(A0), 0, 1023, 0, 16);

  lcd.setCursor(0, 0);
  lcd.print("Count: ");
  lcd.print(count);

  lcd.setCursor(0, 1);
  for (int i = 0; i < 16; i++) {
    if (i < bar) lcd.print("#");
    else lcd.print(" ");
  }
  delay(50);
}
```

1 **&&**

И — оба условия сразу (занятие 8).

2 **map → 0…16**

Ручка 0…1023 превращается в число клеток.

3 **for на 16 клеток**

Закрашиваем **#** или ставим пробел — так стираются старые символы.

Миссия: табло

Высшая ИТ-школа

МИССИЯ

1. Соберите экран, кнопку и потенциометр по карте
2. Выведите приветствие — проверьте, что экран работает
3. Сделайте **счётчик**: одно нажатие = +1
4. Сделайте **шкалу** во второй строке от потенциометра

Готово, если:

- Экран показывает текст
- Счётчик растёт на 1 за нажатие
- Шкала растёт и уменьшается без «хвостов»

★ Усложнение

Вместо шкалы — слово: LOW, MID или HIGH в зависимости от ручки. Или вторая кнопка, которая сбрасывает счётчик в 0.

Дебаггинг: ищем ошибки

Высшая IT-школа

Экран пустой, хотя подсветка горит	Неверный адрес. Попробуйте `0x3F` вместо `0x27` (адрес видно в свойствах детали).
Ошибка: 'class LiquidCrystal_I2C' has no member 'init'	Подключена другая библиотека. Нужна **LiquidCrystal I2C** (Marco Schwartz) или замените `init()` на `begin()`.
Вместо букв — кракозябры	На экране только латиница. Никаких русских букв в `lcd.print`.
Счётчик прыгает на 10–100 за нажатие	Нет проверки `lastButton` — считаете каждый `LOW`, а не момент нажатия.
Цифры «растут хвостом»: 990 вместо 99	Допечатайте пробелы после числа: `lcd.print("  ");`

Итоги и рефлексия

Высшая ИТ-школа

Сегодня мы:

- Подключили **экран LCD 16×2** по шине I2C — 4 провода
- Научились подключать **библиотеки** в Cirkit
- Освоили `setCursor`, `print`, `clear`
- Поймали **момент нажатия** через `lastButton`
- Собрали **табло** со счётчиком и шкалой

1. **Что вам понравилось в работе?**
2. **Что нового было самым важным?**
3. **Что вызвало наибольшие сложности?**

Высшая ИТ-школа

КГУ Костромской государственный университет

12

Метеостанция: датчик температуры и влажности

DHT22 · float · else if · isnan · || · RGB

Занятие 12

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Как работает датчик **DHT22**
- Что такое **дробные числа** — тип `float`
- Как проверить, что датчик ответил: `isnan`
- Лестница решений: `if` / `else if` / `else`
- Как спрятать RGB в функцию `setColor`

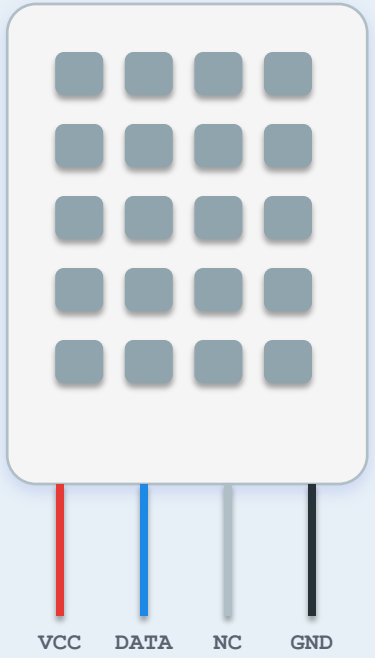
✓ В результате урока

- Соберёте **метеостанцию**: температура и влажность на экране, RGB-индикатор синий / зелёный / красный

Знакомьтесь: датчик DHT22

Высшая ИТ-школа

- Измеряет **температуру**: от −40 до +80 °C
- И **влажность** воздуха: от 0 до 100 %
- Отвечает по **одному** проводу DATA
- Медленный: не чаще **раза в 2 секунды**
- Младший брат **DHT11** — дешевле, но менее точный



DHT22

Внутри:

термистор — сопротивление меняется от тепла

датчик влажности — плёнка впитывает влагу из воздуха

свой микрочип, который всё считает

Новый тип данных: float

Высшая ИТ-школа

int — целые числа

- 10, 500, −3
- Подходит для номеров пинов, счётчиков
- `int t = 23.7;` → в коробке будет **23**
- Дробная часть **отбрасывается**!

vs

float — дробные числа

- 23.7, −0.5, 3.14
- Подходит для температуры, расстояния
- `float t = 23.7;` → в коробке **23.7**
- Дробь пишем через **точку**, а не запятую
- `lcd.print(t, 1)` — 1 знак после точки

Библиотека DHT: читаем погоду

Высшая ИТ-школа

dht_test.ino

```ino
#include <DHT.h>

DHT dht(4, DHT22);    // пин 4, модель DHT22

void setup() {
  dht.begin();
  Serial.begin(9600);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  Serial.print(t);
  Serial.print(" C, ");
  Serial.print(h);
  Serial.println(" %");
  delay(2000);
}
```

1. **Подключение**

   Libraries → «DHT sensor library» (Adafruit) → **Include**.

2. **DHT dht(4, DHT22)**

   Создаём датчик: на каком пине и какой модели.

3. **readTemperature()**

   Функция-ответчик: возвращает **float** — градусы.

4. **delay(2000)**

   Датчик медленный: чаще 2 секунд спрашивать нельзя.

Если датчик молчит: isnan и ||

Высшая ИТ-школа

```
if (isnan(t) || isnan(h)) {
  lcd.setCursor(0, 0);
  lcd.print("Sensor error!");
  delay(2000);
  return;    // начать loop заново
}
```

1. **isnan**

   is NaN — «это не число?». Датчик не ответил — вместо числа пустота.

2. **||**

   ИЛИ: хватит одного «да». Помните `&&` (И) с занятия 8? Там нужны оба.

3. **return**

   Как на занятии 7 с RFID: «выходим и пробуем снова».

4. **Зачем?**

   Иначе на экране будет `nan` — непонятно и некрасиво.

Лестница решений: else if

Высшая ИТ-школа

- Когда вариантов **больше двух**, ставим ступеньки
- Плата проверяет **сверху вниз**
- Первое «да» — выполняет и **дальше не смотрит**
- `else` в конце — «во всех остальных случаях»
- Порядок ступенек **важен**!

```
if (t < 18)
```
→ **Синий: холодно**

```
else if (t <= 26)
```
→ **Зелёный: комфорт**

```
else
```
→ **Красный: жарко**

RGB в функции: setColor

Высшая IT-школа

setcolor.ino

```ino
int redPin = 3;    // только пины с ~
int greenPin = 5;
int bluePin = 6;

void setColor(int r, int g, int b) {
  analogWrite(redPin, r);
  analogWrite(greenPin, g);
  analogWrite(bluePin, b);
}

// использование:
setColor(0, 0, 255);     // синий
setColor(0, 255, 0);     // зелёный
setColor(255, 255, 0);   // жёлтый
```

1 **Функция с 3 параметрами**

Красный, зелёный, синий: от 0 до 255.

2 **Пины 3, 5, 6**

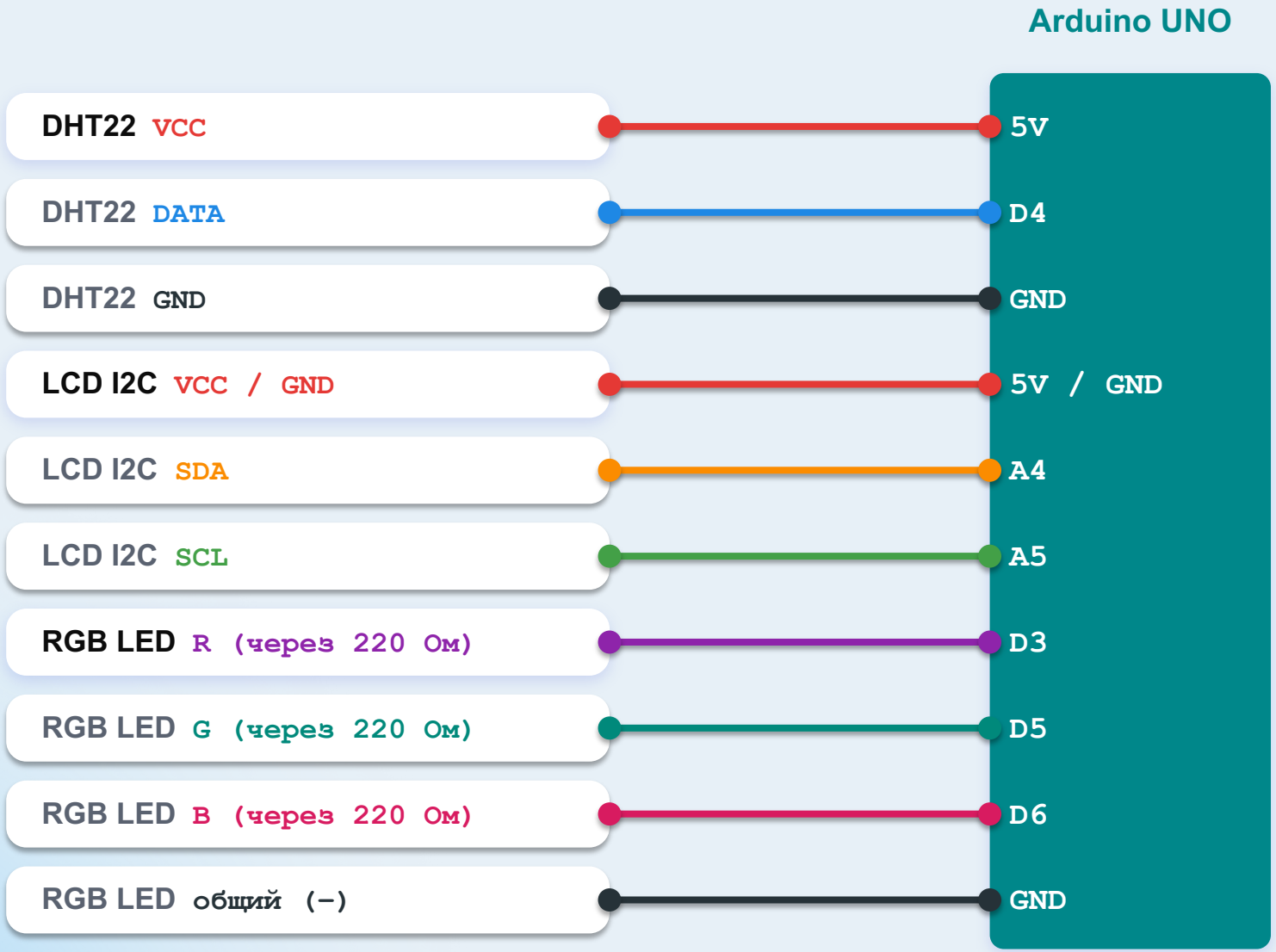
ШИМ-пины с тильдой ~ (занятие 5). Не 9/10/11 — пригодятся серве и другим модулям.

3 **Цвета наоборот?**

RGB с общим анодом: подавайте `255 - r` (в полном коде есть переключатель).

Карта подключения: метеостанция

Высшая ИТ-школа



Как собрать

1. Макетка: шины 5V и GND
2. DHT22: DATA → **D4**
3. Экран — как на прошлом уроке: SDA → A4, SCL → A5
4. RGB: R, G, B через резисторы → **D3, D5, D6**
5. Общий вывод RGB → GND (если общий анод — 5V)

Код метеостанции: главный цикл

Высшая IT-школа

meteo.ino

```
void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (isnan(t) || isnan(h)) { delay(2000); return; }

  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(t, 1);
  lcd.print(" C   ");
  lcd.setCursor(0, 1);
  lcd.print("Hum:  ");
  lcd.print(h, 0);
  lcd.print(" %   ");

  if (t < 18)       setColor(0, 0, 255);
  else if (t <= 26) setColor(0, 255, 0);
  else              setColor(255, 0, 0);
  delay(2000);
}
```

1. **Сверху скетча**

   `#include`, `lcd`, `dht`, `setColor` и `setup` — как на прошлых слайдах.

2. **print(t, 1)**

   Один знак после точки: 23.7

3. **Пробелы**

   Стирают хвосты старых цифр.

4. **Лестница**

   Три цвета — три ступеньки.

Миссия: метеостанция

Высшая ИТ-школа

МИССИЯ

1. Проверьте датчик через **монитор порта**
2. Выведите температуру и влажность на **экран**
3. Добавьте **RGB**: синий / зелёный / красный
4. Испытайте: задайте в симуляции 10, 22 и 35 °C — все три цвета работают?

Готово, если:

- Числа на экране без «хвостов»
- Три цвета по лестнице else if
- При ошибке датчика — надпись Sensor error

★ Усложнение

Добавьте значок градуса: `lcd.print((char)223);`. Или запоминайте максимум температуры и показывайте «Max: 31.5».

Дебаггинг: ищем ошибки

Высшая ИТ-школа

На экране Sensor error или nan	Проверьте провод DATA → **D4** и строку `DHT dht(4, DHT22)` — модель и пин должны совпадать.
Ошибка: DHT.h: No such file	Не подключена библиотека: **Libraries** → DHT sensor library → **Include**.
Температура без дробной части	Переменная объявлена как `int` вместо `float`.
Всегда один цвет	Неправильный порядок ступенек или RGB подключён не к пинам с ~.

Итоги и рефлексия

Высшая ИТ-школа

Сегодня мы:

- Подключили датчик **DHT22** и прочитали погоду
- Узнали тип `float` для дробных чисел
- Научились ловить ошибку датчика: `isnan` и `||`
- Построили **лестницу** `else if`
- Собрали **метеостанцию** с экраном и RGB

1. **Что вам понравилось в работе?**
2. **Что нового было самым важным?**
3. **Что вызвало наибольшие сложности?**

Высшая ИТ-школа

КГУ Костромской государственный университет

13

Движение: сервопривод и шлагбаум

Servo | Угол 0–180° | map() | Плавность | Шлагбаум

Занятие 13

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Чем **сервопривод** отличается от обычного мотора
- Как управлять углом: библиотека `Servo`
- Как сделать движение **плавным**
- Почему серва «конфликтует» с ШИМ на пинах 9 и 10

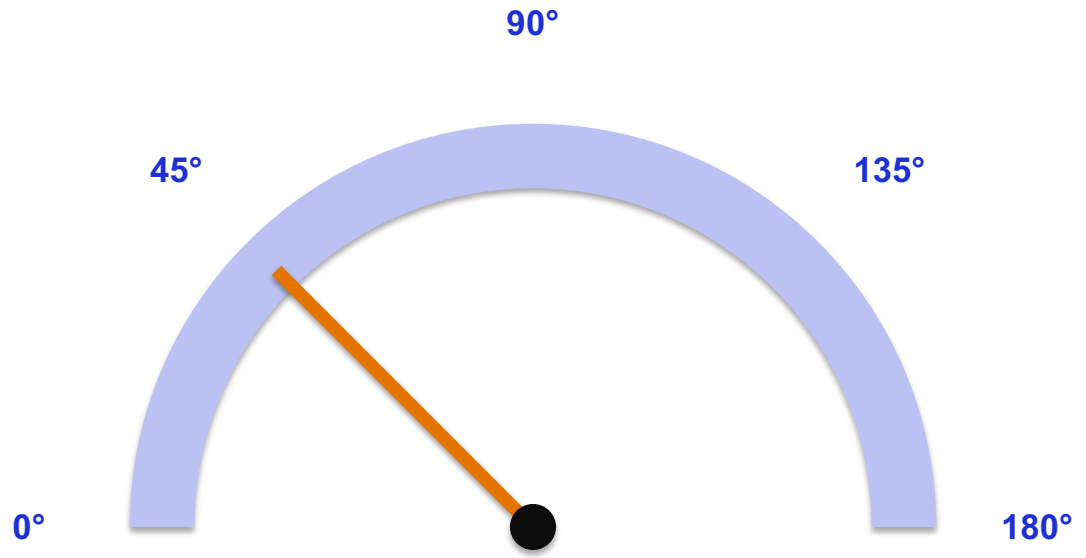
✓ В результате урока

- Повернёте серву **ручкой-потенциометром**
- Соберёте **шлагбаум**, который открывается и закрывается по кнопке

Мотор VS сервопривод

Высшая ИТ-школа

- Обычный мотор: крутится **без остановки**, пока есть ток
- Сервопривод: поворачивается **на точный угол** и держит его
- Диапазон: от **0°** до **180°** — половина круга
- Внутри: мотор + шестерёнки + датчик угла + свой мозг
- Где: шлагбаумы, рули моделей, руки роботов, заслонки



`write(45)` — стрелка встанет на 45°

Три провода сервопривода

Высшая ИТ-школа

- **Коричневый** (или чёрный) — GND
- **Красный** — 5V, питание
- **Оранжевый** (или жёлтый) — сигнал, к любому цифровому пину
- Сигнал — это ШИМ-импульсы: длина импульса = угол
- В жизни мощной серве нужен **отдельный блок питания**

Библиотека Servo

Высшая ИТ-школа

servo_test.ino

```
#include <Servo.h>

Servo myServo;              // создаём серву

void setup() {
  myServo.attach(9);        // сигнал — на пин 9
}

void loop() {
  myServo.write(0);         // 0 градусов
  delay(1000);
  myServo.write(90);        // 90 градусов
  delay(1000);
  myServo.write(180);       // 180 градусов
  delay(1000);
}
```

1 **Устанавливать не нужно**

Servo — встроенная библиотека Arduino. Просто `#include`.

2 **attach(9)**

«Серва, слушай пин 9».

3 **write(угол)**

Повернуться на угол от 0 до 180.

4 **Важно!**

С Servo пины **9 и 10** теряют ШИМ. RGB — на 3, 5, 6.

Разминка: серва слушается ручку

Высшая ИТ-школа

ruchka.ino

```ino
#include <Servo.h>
Servo myServo;

void setup() {
  myServo.attach(9);
}

void loop() {
  int pot = analogRead(A0);
  int angle = map(pot, 0, 1023, 0, 180);
  myServo.write(angle);
  delay(15);
}
```

1 **map снова с нами**

Ручка 0…1023 → угол 0…180. Как с яркостью на занятии 5.

2 **delay(15)**

Даём серве время доехать до нового угла.

3 **Задание**

Добавьте потенциометр на A0 и покрутите ручку во время симуляции.

Плавное движение: серва как лифт

Высшая ИТ-школа

- `write(90)` — серва **прыгает** на 90° максимально быстро
- Настоящий шлагбаум поднимается **плавно**
- Решение: цикл `for` — по 1 градусу с паузой 15 мс
- 90 шагов × 15 мс ≈ **1,4 секунды** на подъём
- Меньше пауза — быстрее движение

```
for (int a = 0; a <= 90; a++) {
  gate.write(a);
  delay(15);
}
```

Шлагбаум: функция moveGate

Высшая ИТ-школа

shlagbaum.ino

```
void moveGate(int from, int to) {
  if (from < to) {
    for (int a = from; a <= to; a++) {
      gate.write(a);
      delay(15);
    }
  } else {
    for (int a = from; a >= to; a--) {
      gate.write(a);
      delay(15);
    }
  }
}
```

1. **Два параметра**

   Откуда и куда ехать.

2. **Вверх или вниз**

   Если `from < to` — считаем вверх (a++), иначе вниз (a--).

3. **Повторное использование**

   Эта же функция пригодится в теплице и гараже на хакатоне!

Шлагбаум: память и кнопка

Высшая ИТ-школа

shlagbaum.ino

```ino
#include <Servo.h>
Servo gate;
int gateOpen = 0;    // 0 — закрыт, 1 — открыт

void setup() {
  gate.attach(9);
  gate.write(0);
  pinMode(2, INPUT_PULLUP);
}

void loop() {
  if (digitalRead(2) == LOW) {
    tone(8, 1500, 100);
    if (gateOpen == 0) { moveGate(0, 90); gateOpen = 1; }
    else               { moveGate(90, 0); gateOpen = 0; }
    delay(300);
  }
}
```

1. **gateOpen**

   Переменная состояния — как в хлопковом ночнике (занятие 6).

2. **Одна кнопка — два действия**

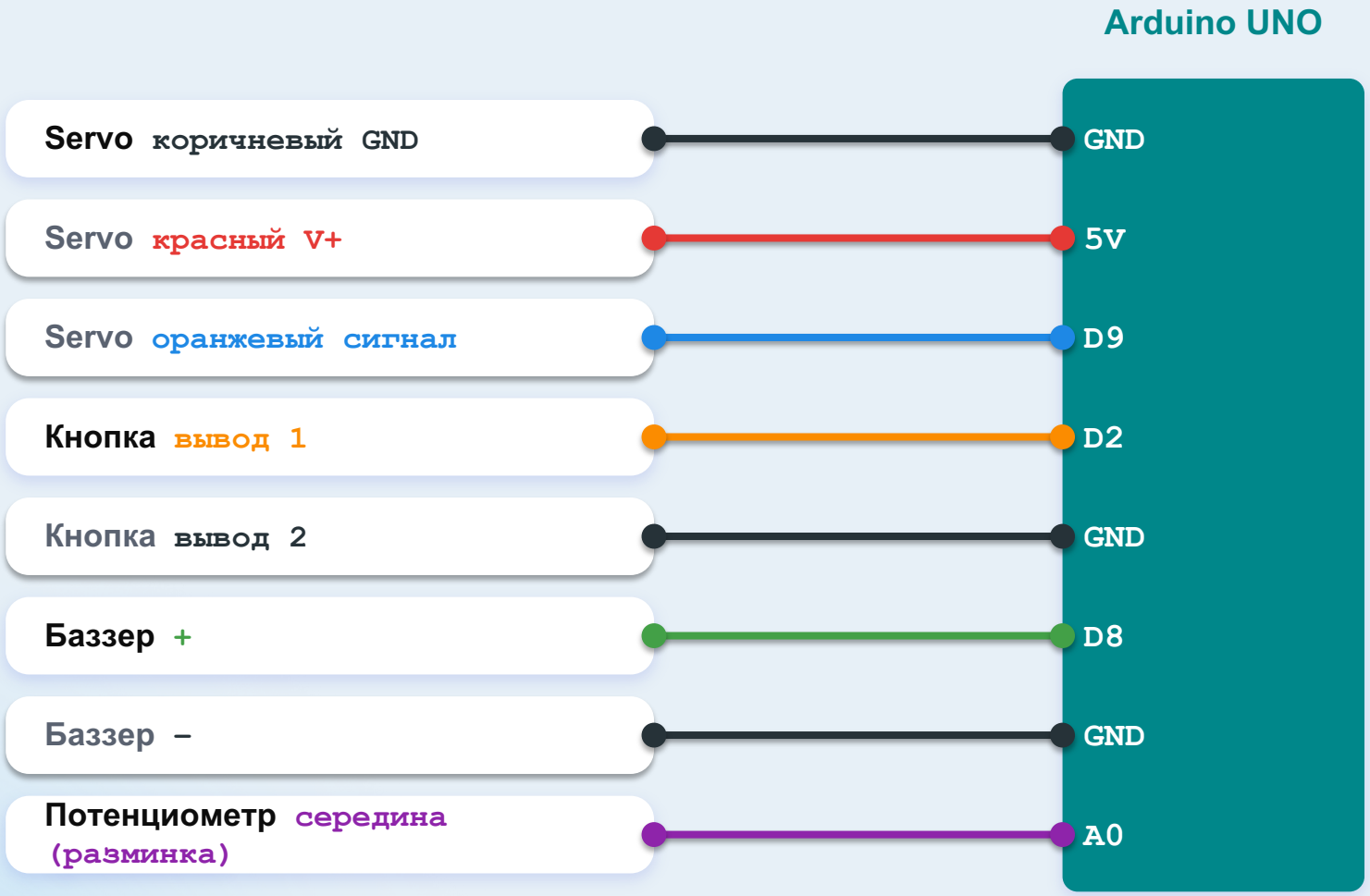
   Шлагбаум помнит, открыт он или закрыт.

3. **tone**

   Короткий «бип» на баззере D8 — подтверждение нажатия.

Карта подключения: шлагбаум

Высшая ИТ-школа



Как собрать

1. Найдите **Servo** в Simulation Parts
2. Сигнал сервы → **D9**
3. Кнопка — D2 и GND, баззер — D8 и GND
4. Потенциометр нужен только для разминки: края 5V и GND, середина A0
5. Серва должна стартовать в положении 0° (закрыто)

Миссия: шлагбаум

Высшая ИТ-школа

МИССИЯ

1. Разминка: серва повторяет **ручку** потенциометра
2. Напишите функцию `moveGate(from, to)`
3. Кнопка: **открыть / закрыть** с памятью `gateOpen`
4. Добавьте «бип» при каждом нажатии

Готово, если:

- Шлагбаум движется плавно
- Одна кнопка и открывает, и закрывает
- Есть функция moveGate

★ Усложнение

Стрелочный термометр: DHT22 + серва. `map((int)t, -10, 40, 180, 0)` — стрелка показывает температуру, как на старых приборах.

Дебаггинг: ищем ошибки

Высшая ИТ-школа

Серва не двигается	Проверьте `attach(9)` и провод сигнала на **D9**. Деталь — из Simulation Parts?
Шлагбаум открывается и сразу закрывается	Нет `delay(300)` после нажатия — кнопка считывается дважды.
Серва крутится «не в ту сторону»	Поменяйте углы местами или используйте `180 - angle`.
RGB на 9 или 10 перестал плавно светить	Servo занимает ШИМ на 9 и 10. Переносим RGB на **3, 5, 6**.

Итоги и рефлексия

Высшая ИТ-школа

Сегодня мы:

- Узнали, чем **сервопривод** отличается от мотора
- Управляли углом через библиотеку `Servo`
- Сделали движение **плавным** с помощью `for`
- Собрали **шлагбаум** с памятью состояния

1. **Что вам понравилось в работе?**
2. **Что нового было самым важным?**
3. **Что вызвало наибольшие сложности?**

Высшая ИТ-школа

КГУ Костромской государственный университет

14

Летучая мышь: ультразвуковой дальномер

HC-SR04 | Эхолокация | pulseIn | return float | Парктроник

Занятие 14

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Как работает **эхолокация**
- Как устроен датчик **HC-SR04**
- Как по времени эха посчитать **расстояние**
- Как написать функцию, которая возвращает `float`

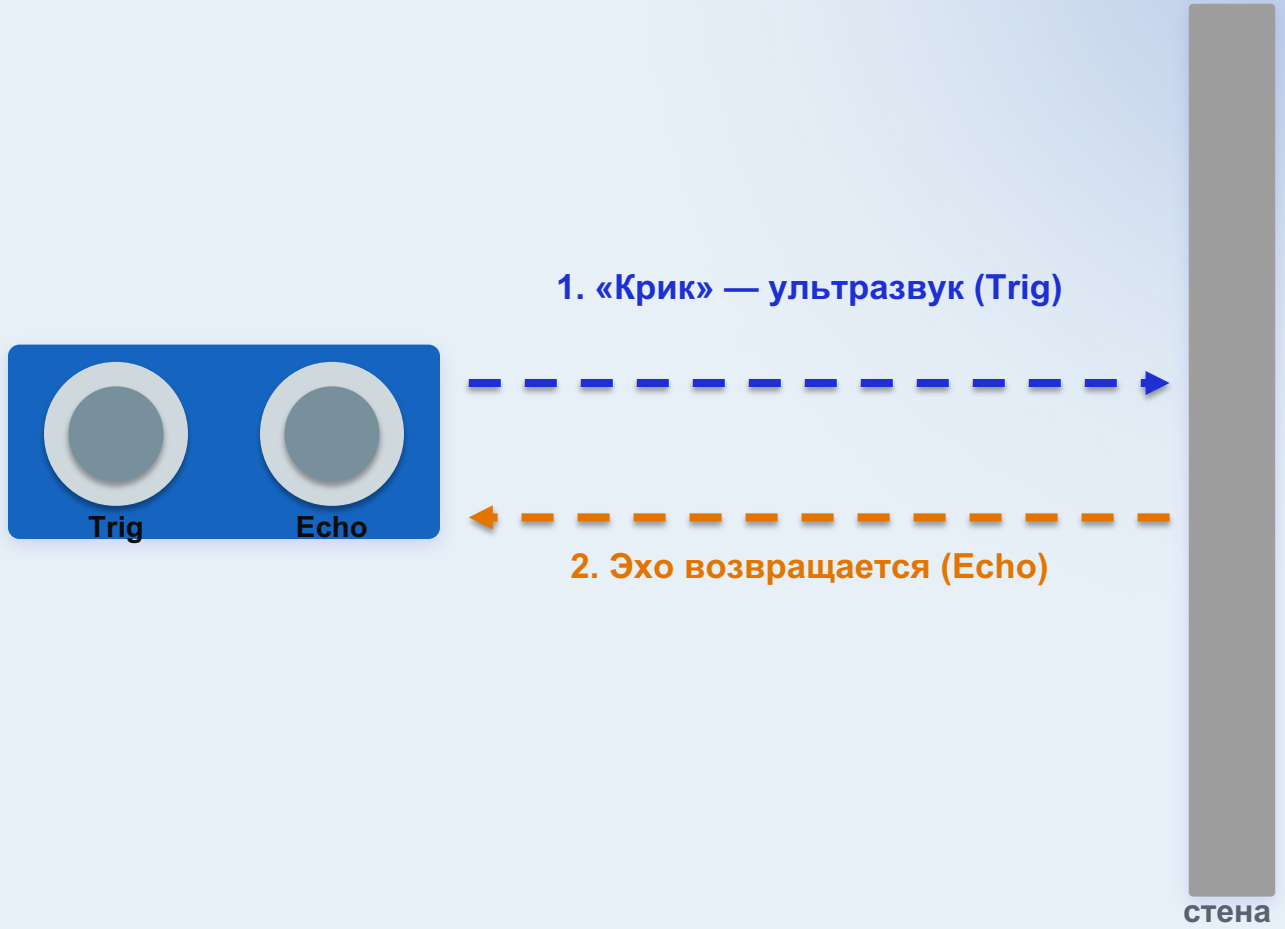
✓ В результате урока

- Соберёте **парктроник**: чем ближе препятствие, тем чаще пищит баззер; расстояние — на экране

Эхолокация: как видит летучая мышь

Высшая ИТ-школа

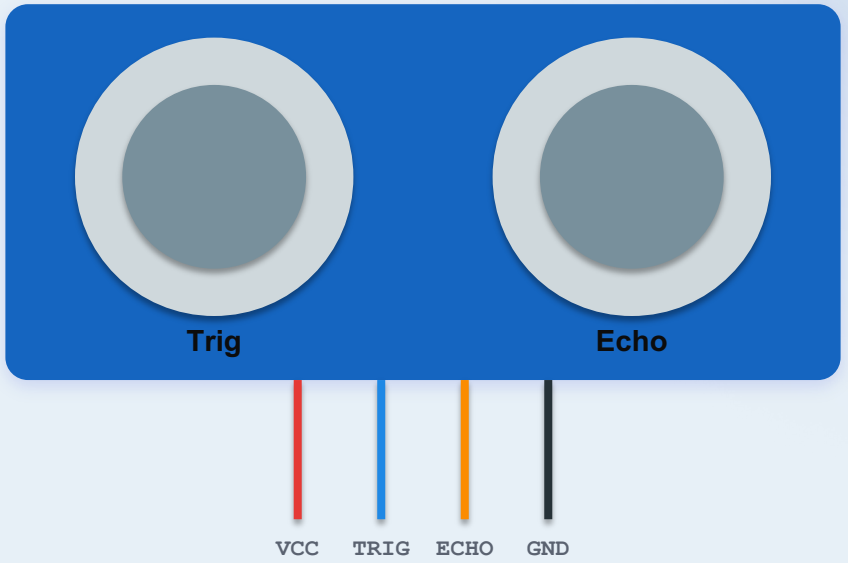
- Мышь **кричит** ультразвуком — мы его не слышим
- Звук летит, **отражается** от стены и возвращается
- Чем дальше стена — тем **дольше** ждать эхо
- Так же «видят» дельфины, подводные лодки и парктроники



Знакомьтесь: HC-SR04

Высшая ИТ-школа

- Два «глаза» — на самом деле **рот и ухо**
- **Trig** — «рот»: кричит, когда получает импульс
- **Echo** — «ухо»: держит HIGH, пока ждёт эхо
- Дальность: от **2 до 400 см**
- 4 вывода: VCC, TRIG, ECHO, GND



Физика: из времени — в сантиметры

Высшая ИТ-школа

```
расстояние = время × 0.0343 / 2
```

0.0343

Скорость звука: 343 м/с = **0,0343 см** за 1 микросекунду

/ 2

Звук летит **туда и обратно** — делим путь пополам

Пример

Эхо пришло через **1000 мкс** → 1000 × 0,0343 / 2 ≈ **17 см**

Функция measureDistance()

Высшая ИТ-школа

parktronik.ino

```
float measureDistance() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);  // крик 10 мкс
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) {
    return 999;   // эхо не вернулось
  }
  return duration * 0.0343 / 2;
}
```

1. **float вместо void**

   Функция **вернёт** дробное число — сантиметры.

2. **delayMicroseconds**

   Пауза в **микросекундах**: 1 мс = 1000 мкс.

3. **pulseIn**

   Ждёт, пока ECHO будет HIGH, и меряет, сколько это длилось.

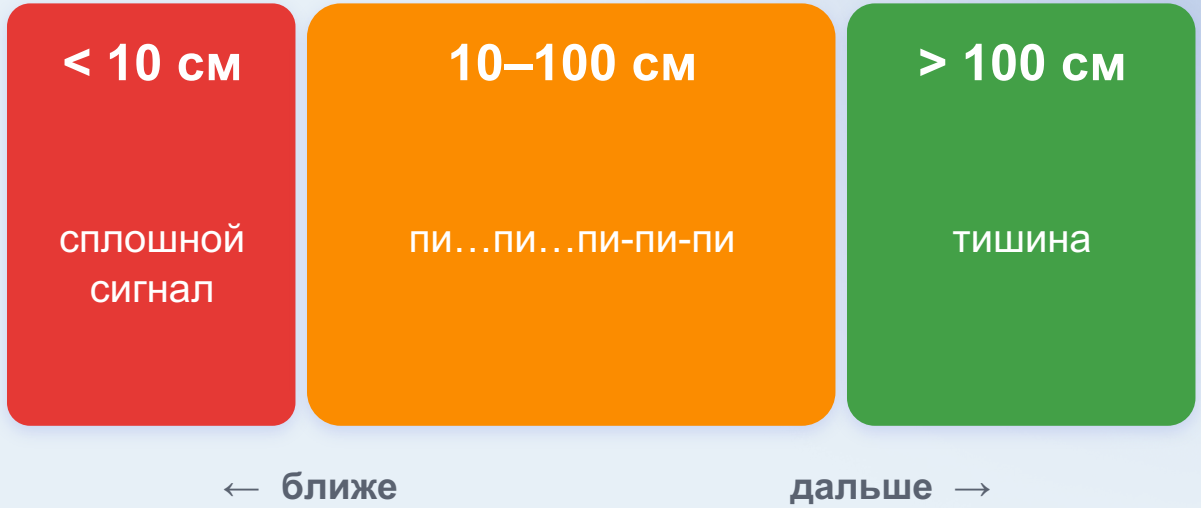
4. **30000**

   Не ждём дольше 30 мс. Нет эха — значит очень далеко: 999.

Логика парктроника

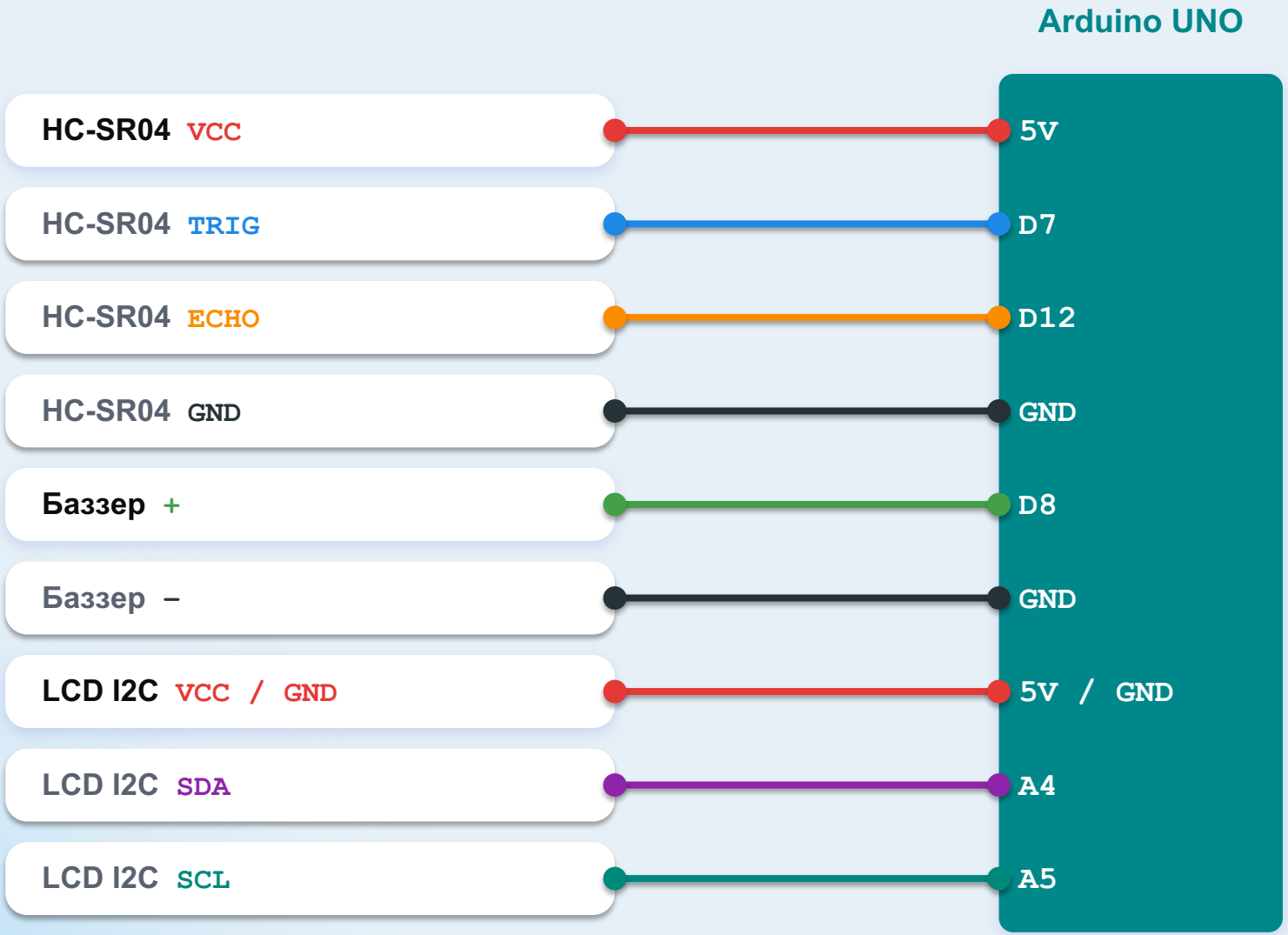
Высшая ИТ-школа

- Дальше **100 см** — тишина
- От **10 до 100 см** — короткие писки, чем ближе, тем чаще
- Ближе **10 см** — сплошной сигнал: СТОП!
- Пауза между писками: `map(cm, 10, 100, 100, 600)`
- Обратите внимание: map может «переворачивать» диапазон



Карта подключения: парктроник

Высшая ИТ-школа



Как собрать

1. HC-SR04 из Simulation Parts
2. TRIG → **D7**, ECHO → **D12**
3. Баззер — D8 (как всегда)
4. Экран — A4 / A5
5. Во время симуляции кликните по датчику и меняйте расстояние

Код парктроника: главный цикл

Высшая IT-школа

parktronik.ino

```
void loop() {
  float cm = measureDistance();
  lcd.setCursor(0, 1);
  lcd.print(cm, 1);
  lcd.print(" cm     ");

  if (cm < 10) {
    tone(buzzerPin, 2000);          // СТОП!
    delay(100);
  } else if (cm < 100) {
    tone(buzzerPin, 1000, 50);      // пик
    delay(map((int)cm, 10, 100, 100, 600));
  } else {
    noTone(buzzerPin);              // тишина
    delay(200);
  }
}
```

1 **(int)cm**

map работает с целыми числами — превращаем float в int.

2 **Три зоны**

Лестница `if` / `else if` / `else`.

3 **Сверху скетча**

`#include`, `lcd`, пины и `setup` с `pinMode` — полный код в сценарии.

Миссия: парктроник

Высшая ИТ-школа

МИССИЯ

1. Соберите датчик и выведите расстояние в **монитор порта**
2. Напишите функцию `measureDistance()`
3. Выведите расстояние на **экран**
4. Добавьте **три зоны** звука

Готово, если:

- Расстояние правдоподобное
- Писк ускоряется при приближении
- Ближе 10 см — сплошной сигнал

★ Усложнение

Добавьте RGB-светофор: зелёный, жёлтый, красный по зонам. Или шкалу на второй строке экрана, как на занятии 11.

Дебаггинг: ищем ошибки

Высшая ИТ-школа

Всегда 999	Перепутаны TRIG и ECHO или в коде другие пины. Проверьте `pinMode`: TRIG — OUTPUT, ECHO — INPUT.
Расстояние «скачет»	В демо Cirkit перед `pulseIn` ставят `noInterrupts()`, а после — `interrupts()`. Попробуйте так.
Писк не меняется	Проверьте `map` и `(int)cm`. Выведите паузу в монитор порта.
Ошибка: 'measureDistance' was not declared	Функция должна быть написана **выше** `loop` или проверьте опечатку.

Итоги и рефлексия

Высшая ИТ-школа

Сегодня мы:

- Разобрались, как работает **эхолокация**
- Посчитали расстояние по **скорости звука**
- Написали функцию, которая возвращает `float`
- Собрали **парктроник** с тремя зонами

1. **Что вам понравилось в работе?**
2. **Что нового было самым важным?**
3. **Что вызвало наибольшие сложности?**

Высшая ИТ-школа

КГУ Костромской государственный университет

15

Многозадачность: millis() вместо delay()

millis() | unsigned long | bool | Таймеры | Парктроник 2.0

Занятие 15

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Почему `delay()` **мешает** делать несколько дел
- Что такое **millis()** — часы платы
- Формула неблокирующего **таймера**
- Типы `unsigned long` и `bool`, оператор `!`

✓ В результате урока

- Мигалка + кнопка, которая **мгновенно** реагирует
- **Парктроник 2.0**: замер, писк, экран и кнопка «тишина» — одновременно

Эксперимент: кнопка «не слышит»

Высшая ИТ-школа

problema.ino

```
void loop() {
  digitalWrite(ledPin, HIGH);
  delay(1000);       // плата спит
  digitalWrite(ledPin, LOW);
  delay(1000);       // и ещё спит

  if (digitalRead(buttonPin) == LOW) {
    tone(buzzerPin, 1000, 200);
  }
}
```

1 Задание

Соберите: светодиод D10, кнопка D2, баззер D8. Запустите.

2 Нажмите кнопку

Быстро. Пищит? А если ещё раз?

3 Почему?

Кнопку проверяем **раз в 2 секунды** — в остальное время плата спит в `delay`.

Варим кашу: delay VS millis

Высшая ИТ-школа

delay() — уснуть у плиты

- Поставил кашу, лёг спать на 10 минут
- В это время позвонили в дверь — не услышал
- Кот уронил чашку — не заметил
- Плата **блокирована**

vs

millis() — поглядывать на часы

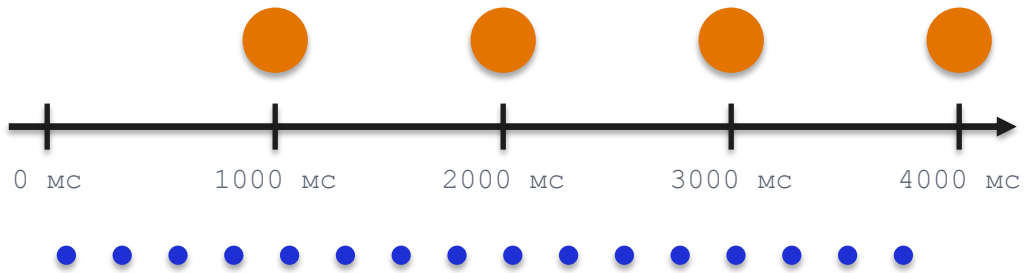
- Поставил кашу, запомнил время
- Делаешь уроки, открываешь дверь, кормишь кота
- Иногда смотришь: прошло 10 минут? — снимаешь кашу
- Плата **свободна** между проверками

millis() — часы платы

Высшая ИТ-школа

- `millis()` возвращает, сколько **миллисекунд** прошло с включения платы
- Через 1 секунду — 1000, через минуту — 60 000…
- Числа огромные — нужна большая коробка: `unsigned long`
- `unsigned` — **без минуса**: от 0 до 4 млрд
- Хватает на ~49 дней работы без перерыва

каждые 1000 мс: «пора!» → переключаем светодиод



а между ними loop крутится тысячи раз и проверяет кнопку

Формула таймера

Высшая IT-школа

```
if (millis() - lastTime >= 1000) {
  lastTime = millis();
  // делаем дело
}
```

`millis() - lastTime` — Сколько прошло **с прошлого раза**

`>= 1000` — Прошла секунда? Пора!

`lastTime = millis()` — Запоминаем время — **заводим будильник снова**

Правило: в loop **нет ни одного** длинного `delay`.

Каждое дело проверяет **свой** таймер — и loop крутится тысячи раз в секунду, успевая всё.

Решение: мигалка + мгновенная кнопка

Высшая ИТ-школа

millis.ino

```
unsigned long lastBlink = 0;
bool ledOn = false;

void loop() {
  // Дело 1: мигать раз в секунду
  if (millis() - lastBlink >= 1000) {
    lastBlink = millis();
    ledOn = !ledOn;
    digitalWrite(ledPin, ledOn);
  }

  // Дело 2: пищать, пока нажата кнопка
  if (digitalRead(buttonPin) == LOW) {
    tone(buzzerPin, 1000);
  } else {
    noTone(buzzerPin);
  }
}
```

1 **bool**

Коробка для **да / нет**: `true` или `false`.

2 **!ledOn**

НЕ: переворачивает true ↔ false. Включено → выключено.

3 **Проверьте**

Теперь кнопка пищит **мгновенно**, а светодиод продолжает мигать.

Много дел — много таймеров

Высшая ИТ-школа

1

Замер расстояния

Свой таймер: **каждые 100 мс**

2

Писк баззера

Свой таймер: пауза зависит от расстояния

3

Экран

Свой таймер: **каждые 300 мс** (чаще — мерцает)

4

Кнопка «тишина»

Без таймера — проверяем **каждый** проход loop

Парктроник 2.0: таймеры

Высшая ИТ-школа

parktronik2.ino

```
unsigned long lastMeasure = 0, lastBeep = 0, lastScreen = 0;
float cm = 999;
bool mute = false;
int lastButton = HIGH;

void loop() {
  unsigned long now = millis();

  if (now - lastMeasure >= 100) {      // 1. замер
    lastMeasure = now;
    cm = measureDistance();
  }

  int button = digitalRead(buttonPin);  // 2. кнопка
  if (button == LOW && lastButton == HIGH) mute = !mute;
  lastButton = button;
  // 3. писк и 4. экран — на следующем слайде
```

1 **now**

Время берём один раз за проход.

2 **mute = !mute**

Кнопка переключает «тишину» — момент нажатия, как на занятии 11.

Парктроник 2.0: писк и экран

Высшая ИТ-школа

parktronik2.ino

```
  if (mute || cm >= 100) {                  // 3. писк
    noTone(buzzerPin);
  } else if (cm < 10) {
    tone(buzzerPin, 2000);
  } else {
    unsigned long pause = map((int)cm, 10, 100, 100, 600);
    if (now - lastBeep >= pause) {
      lastBeep = now;
      tone(buzzerPin, 1000, 50);
    }
  }

  if (now - lastScreen >= 300) {            // 4. экран
    lastScreen = now;
    lcd.setCursor(0, 1);
    lcd.print(cm, 1);
    lcd.print(mute ? " cm MUTE " : " cm      ");
  }
}
```

1. **Таймер с переменной паузой**
   `pause` зависит от расстояния — и всё равно без delay!

2. **? :**
   Короткий if: «если mute — одно, иначе — другое».

Миссия: многозадачность

Высшая ИТ-школа

МИССИЯ

1. Переделайте «проблемную» программу: мигалка на `millis()`, кнопка — мгновенно
2. Добавьте **второй** светодиод, мигающий раз в 300 мс
3. Возьмите парктроник с прошлого урока и уберите все `delay`
4. Добавьте кнопку «тишина»

Готово, если:

- В loop нет длинных delay
- Кнопка реагирует мгновенно
- Два светодиода мигают в разном ритме

★ Усложнение

Парктроник 2.0 целиком: писк + экран + кнопка «тишина» + RGB-светофор по зонам — всё одновременно.

Дебаггинг: ищем ошибки

Высшая ИТ-школа

Таймер срабатывает один раз и всё	Забыли обновить: `lastTime = millis();` внутри if.
Через ~30 секунд всё ломается	Таймер объявлен как `int` — нужно `unsigned long`.
Всё равно тормозит	В loop остался длинный `delay`. Найдите и замените таймером.
Светодиод горит, но не мигает	Нет `ledOn = !ledOn;` — состояние не переворачивается.

Итоги и рефлексия

Высшая ИТ-школа

Сегодня мы:

- Увидели, почему `delay` **блокирует** плату
- Узнали часы платы: `millis()` и `unsigned long`
- Запомнили **формулу таймера**
- Научились делать **несколько дел** одновременно
- Собрали **парктроник 2.0**

1. **Что вам понравилось в работе?**
2. **Что нового было самым важным?**
3. **Что вызвало наибольшие сложности?**

Высшая ИТ-школа

КГУ Костромской государственный университет

16

Финальный хакатон: умный город

Команды | Алгоритм | Сборка | Испытания | Защита

Занятие 16

Что узнаем и что сделаем

Высшая ИТ-школа

? Что узнаем

- Как объединить **несколько модулей** в одну систему
- Как распределить роли в команде
- Как защищать свой проект

✓ В результате урока

- Соберёте и защитите **проект** «умного города»
- Заполните **финальную анкету** и увидите свой прогресс

Разведка: наш арсенал

Высшая ИТ-школа

Кнопка

`INPUT_PULLUP` · D2

Баззер

`tone()` · D8

RGB

`setColor()` · D3, D5, D6

Экран LCD

I2C · A4, A5

DHT22

температура · D4

Сервопривод

`moveGate()` · D9

HC-SR04

`measureDistance()` · D7, D12

Клавиатура 4×4

`Keypad` · D4–D7, D10–D13

Проект 1: Умная теплица

Теплица сама следит за климатом и проветривается.

Высшая ИТ-школа

Модули

- DHT22
- Серво-форточка
- LCD
- RGB
- Баззер

Логика

- Жарче **28 °C** → форточка открывается (серва 90°)
- Холоднее **25 °C** → закрывается
- RGB: синий / зелёный / красный
- Жарче **35 °C** → тревога баззером
- На экране: T, H и «Window: OPEN / CLOSED»

Пины: DHT D4 · Серво D9 · RGB D3/D5/D6 · Баззер D8 · LCD A4/A5

Проект 2: Умный гараж

Высшая ИТ-школа

Шлагбаум сам видит машину, а парктроник помогает встать.

Модули

- HC-SR04
- Серво-шлагбаум
- LCD
- RGB
- Баззер

Логика

- Машина ближе **30 см** → «Welcome!» и шлагбаум вверх
- Уехала дальше **50 см** → шлагбаум вниз
- RGB: зелёный — открыто, красный — закрыто
- Ближе **10 см** при открытом — писк «стоп»
- На экране — расстояние

Пины: TRIG D7 · ECHO D12 · Серво D9 · RGB D3/D5/D6 · Баззер D8 · LCD A4/A5

Проект 3: Сейф 2.0

Высшая ИТ-школа

Сейф с кодом на клавиатуре и сиреной от взломщиков.

Модули

- Клавиатура 4×4
- Серво-замок
- LCD
- Баззер

Логика

- Цифры кода на экране — **звёздочками**
- `#` — проверить, `*` — стереть
- Код верный → замок 90° на 3 секунды
- 3 ошибки подряд → **сирена**
- Код хранится в переменной `secret`

Пины: Ряды D13/D12/D11/D10 · Колонки D7/D6/D5/D4 · Серво D9 · Баззер D8 · LCD A4/A5

Подсказка 1: «окно» против дёрганья

Высшая ИТ-школа

- Если открывать при **> 28**, а закрывать при **< 28**, на 28,0° форточка будет **дёргаться**
- Решение: два порога с «окном» между ними
- Открыть — при **> 28**, закрыть — только при **< 25**
- Между 25 и 28 — **ничего не делаем**, ждём
- Так же работают кондиционеры и холодильники

```
if (t > 28 && windowOpen == 0) {
  moveWindow(0, 90);
  windowOpen = 1;
} else if (t < 25 && windowOpen == 1) {
  moveWindow(90, 0);
  windowOpen = 0;
}
```

Подсказка 2: клавиатура 4×4

Высшая ИТ-школа

keypad_test.ino

```ino
#include <Keypad.h>

char keys[4][4] = {
  {'1','2','3','A'}, {'4','5','6','B'},
  {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[4] = {13, 12, 11, 10};
byte colPins[4] = {7, 6, 5, 4};
Keypad keypad = Keypad(makeKeymap(keys),
                       rowPins, colPins, 4, 4);

void loop() {
  char key = keypad.getKey();
  if (key != NO_KEY) {
    Serial.println(key);
  }
}
```

1. **Библиотека**

   Libraries → Keypad → **Include**.

2. **char**

   Тип для **одного символа**: '1', '#', 'A'.

3. **Таблица кнопок**

   Двумерный массив, как цифры в занятии 10.

4. **getKey()**

   Возвращает нажатую кнопку или `NO_KEY`.

Подсказка 3: switch / case

Высшая ИТ-школа

seif.ino

```
switch (key) {
  case '*':              // стереть
    entered = "";
    break;
  case '#':              // проверить
    if (entered == secret) {
      lock.write(90);    // открыть
    }
    entered = "";
    break;
  default:               // цифра — добавить
    entered += key;
}
```

1. **switch**

   «Посмотри, что в key». Удобнее длинной лестницы if.

2. **case**

   Вариант. `break` — «выходим, дальше не смотрим».

3. **default**

   Всё остальное — как `else`.

4. **String**

   Строка текста: `entered += key` добавляет символ в конец.

План хакатона

Высшая ИТ-школа

1

Команды и выбор

10 мин. Команды по 2–3 человека, выбираем проект и распределяем роли

2

Алгоритм

15 мин. На листе: какие модули, какие пины, что делает устройство — по шагам

3

Сборка и код

40 мин. Собираем по модулю и проверяем каждый в мониторе порта

4

Испытания

15 мин. Проверяем все режимы, приводим провода в порядок

5

Защита

10 мин. 2 минуты на команду + финальная анкета

Стратегия «по кусочкам»: датчик → монитор порта → экран → исполнитель (серва, баззер) → логика. Каждый шаг проверяем, прежде чем идти дальше.

Роли в команде

Высшая ИТ-школа

1

Инженер схемы

Собирает схему в Cirkit по карте пинов. Следит за цветами проводов. Проверяет каждое подключение.

2

Программист

Пишет код по алгоритму. Начинает с функций для каждого модуля. Проверяет в мониторе порта.

3

Тестировщик и спикер

Придумывает испытания, ищет ошибки, готовит рассказ для защиты.

Защита проекта: 2 минуты

Высшая ИТ-школа

1

Покажите

Устройство в работе — во **всех режимах**

2

Объясните

Самую сложную часть кода: как она работает

3

Расскажите

Где это пригодится **в жизни**

4

Бонус

Своя «фишка», которой не было в задании

Финальная анкета и что дальше

Высшая ИТ-школа

1

Анкета знаний

Заполняем ту же анкету, что на занятии 1. Сравниваем: что вы умели **до** и **после**?

2

Настоящая плата

Любой проект модуля 2 можно собрать на реальном Arduino — код **не меняется**.

3

Что изучать дальше

ESP32 и Wi-Fi, умный дом через интернет, роботы на моторах, 3D-печать корпусов.

Высшая ИТ-школа

КГУ Костромской государственный университет

16/16

Вы — инженеры виртуальной лаборатории

Функции · LCD · DHT22 · Servo · HC-SR04 · millis() · Хакатон

Спасибо за модуль 2!